

7

Expressions and Assignment Statements

- 7.1 Introduction
- 7.2 Arithmetic Expressions
- 7.3 Overloaded Operators
- 7.4 Type Conversions
- 7.5 Relational and Boolean Expressions
- 7.6 Short-Circuit Evaluation
- 7.7 Assignment Statements
- 7.8 Mixed-Mode Assignment

Türkçe Anlatım · Terimler İngilizce

Konu Anlatımı + Review Questions (27) + Problem Set (21)

Gebze Technical University · Spring 2026

İçindekiler

1 Giriş (Introduction)	2
1.1 İmperative Diller ve Assignment'ın Merkezi Rolü	2
1.2 Temel Tasarım Soruları	3
2 Arithmetic Expressions	4
2.1 Operator Precedence (Operatör Önceliği)	4
2.2 Operator Associativity (Birleştirme Yönü)	4
2.3 Parentheses (Parantezler)	5
2.4 Özel Dil Yaklaşımları	5
2.5 Operand Evaluation Order (Operand Değerlendirme Sırası)	6
2.6 Referential Transparency (Şeffaf Referans)	7
3 Overloaded Operators	9
3.1 Tehlikeli Overloading: C++'da & Operatörü	9
3.2 User-Defined Operator Overloading	9
4 Type Conversions (Tip Dönüşümleri)	11
4.1 Widening ve Narrowing Conversions	11
4.2 Coercion in Expressions	11
4.3 Explicit Type Conversion (Cast)	12
5 Relational and Boolean Expressions	13
5.1 Relational Expressions	13
5.2 Boolean Expressions	14
6 Short-Circuit Evaluation	15
6.1 Pratik Kullanım: Dizi Sınır Kontrolü	15
6.2 Short-Circuit + Side Effect = Tehlikeli Kombinasyon	15
7 Assignment Statements	17
7.1 Simple Assignment (Basit Atama)	17
7.2 Assignment Çeşitleri	17
7.3 Compound Assignment Operators	17
7.4 Unary Assignment Operators — ++ ve --	18
7.5 Assignment as an Expression	18
7.6 Multiple Assignments (Perl, Ruby)	19
7.7 Assignment in Functional Programming	19
8 Mixed-Mode Assignment	20
9 Review Questions — Soru Çözümleri	22
10 Problem Set — Soru Çözümleri	28

1 Giriş (Introduction)

Programlama dillerinde bir **expression (ifade)** değerlendirebilmek, o dilin **semantiğini** anlamak demektir. Bu bölümde expression'ların nasıl değerlendirildiğini, hangi kuralların sırayı belirlediğini ve bu kuralların neden dil tasarımında kritik önem taşıdığını öğreneceğiz.

Tanım: Expression (İfade) Nedir?

Bir **expression**, programlama dilinde hesaplama belirtmenin temel yoludur. Her expression şu bileşenlerden oluşabilir:

- **Operatörler (operators):** İşlem sembolleri (+, -, *, /, vb.)
- **Operandlar (operands):** Değişkenler, sabitler, fonksiyon çağrıları
- **Parantezler:** Değerlendirme sırasını değiştirmek için

Bir expression'ın değeri; operatörlerin **hangi sırada** değerlendirildiğine ve operandların **ne zaman** elde edildiğine bağlıdır.

Neden Bu Konu Önemlidir?

Aynı expression farklı dillerde — hatta aynı dilin farklı derleyicilerinde — farklı sonuç verebilir. Bunun nedenleri:

- **Operator precedence** kuralları dilden dile değişir.
- **Operand evaluation order** çoğu dilde tanımlanmamıştır.
- **Side effect** içeren fonksiyonlar bu belirsizliği gerçek hatalara dönüştürür.
- Tip dönüşüm kuralları (**coercion**) hataları gizleyebilir.

Tarihsel Not: Expressions Nereden Geliyor?

Programlama dillerindeki aritmetik expression'ların büyük çoğunluğu, matematik ve mühendislikte yüzyıllardır kullanılan **gösterim kurallarından** (notation conventions) miras alınmıştır.

İlk yüksek seviyeli dil **Fortran** (1957), matematiksel ifadeleri otomatik değerlendirebilme hedefiyle tasarlandı. Assembly programlamaya kıyasla devrimciydi: artık programcılar **nasıl hesaplandığını** değil, **ne hesaplandığını** yazabiliyordu.

1.1 Imperative Diller ve Assignment'ın Merkezi Rolü

İmperative (zorunlu) programlama dillerinin özü, **assignment statement (atama ifadesi)**'dir. Bir atama ifadesinin amacı, değişkenlerin değerlerini değiştirerek programın **durumunu (state)** güncellemektir. Başka bir deyişle: **değişkenler zaman içinde değer değiştirir**; bu değişimi sağlayan mekanizma atamalardır.

Fonksiyonel dillerde ise durum farklıdır: değişkenler fonksiyon parametreleri gibi davranır ve bir kez bağlandıktan (bind) sonra değerleri değişmez. Bu nedenle fonksiyonel dillerde “assignment” kavramı yerine **name binding (isim bağlama)** kullanılır.

1.2 Temel Tasarım Soruları

#	Tasarım Sorusu	İlgili Alt Konu
1	Operatör öncelik kuralları nelerdir?	7.2.1 Precedence
2	Operatör birleştirme yönü nedir?	7.2.1 Associativity
3	Operandlar hangi sırayla değerlendirilir?	7.2.2 Evaluation Order
4	Side effect'lere kısıtlama var mı?	7.2.2 Side Effects
5	Kullanıcı operatör overloading yapabilir mi?	7.3 Overloaded Ops
6	Tip karışımına izin var mı?	7.4 Type Conversions
7	Boolean kısa devre değerlendirmesi var mı?	7.6 Short-Circuit
8	Assignment kaç farklı biçimde yazılabilir?	7.7 Assignment Stmts

2 Arithmetic Expressions

Aritmetik expression'lar her programlama dilinin temel yapı taşıdır. Bir expression'ın doğru değerlendirilmesi için iki kritik kural seti gereklidir: **precedence (öncelik)** ve **associativity (birleştirme yönü)**.

2.1 Operator Precedence (Operatör Önceliği)

Tanım: Operator Precedence

Operator precedence, farklı öncelik düzeylerine sahip operatörlerin bir expression içinde hangi sırada değerlendirileceğini belirler.

Örnek: $a + b * c$ — $a = 3, b = 4, c = 5$ ise:

- Toplama önce: $(3 + 4) \times 5 = 35$ — matematiksel sezgiye **aykırı**
- Çarpma önce: $3 + (4 \times 5) = 23$ — precedence kuralına **uygun**

Matematikçiler yüzyıllar önce operatörleri bir *hiyerarşiye* yerleştirdi. Çarpma, toplamadan daha öncelikli kabul edilir. Programlama dilleri bu geleneği devralmıştır: matematik formülleri, parantez eklenmeden doğrudan koda yazılabilir.

Öncelik	Ruby	C / C++ / Java	Ada / Fortran
En Yüksek	**	postfix ++, --	** (Fortran)
	unary +, -	prefix ++, --, unary +/ -	unary +/ -
Orta	*, /, %	*, /, %	*, /
En Düşük	binary +, -	binary +, -	binary +, -

APL İstisnası

APL dilinde **tek bir öncelik düzeyi** vardır; tüm operatörler eşit önceliktedir. Değerlendirme sırası tamamen **sağdan sola associativity** ile belirlenir. Bu nedenle $A * B + C$ ifadesi $A * (B + C)$ olarak değerlendirilir — matematiksel beklentiyle çelişir! APL'nin tasarımcısı Ken Iverson bu yaklaşımı bilinçli olarak seçmiştir.

2.2 Operator Associativity (Birleştirme Yönü)

Tanım: Operator Associativity

Associativity, aynı öncelik düzeyindeki iki *bitişik* operatörün hangi sırayla değerlendirileceğini belirler. “Bitişik” demek, iki operatörün arasında tek bir operand olması demektir.

- **Sol associativity (left-to-right):** Soldaki önce.
Örnek: $a - b + c \rightarrow (a - b) + c$

- **Sağ associativity (right-to-left):** Sağdaki önce.
Örnek (Fortran): $A ** B ** C \rightarrow A ** (B ** C)$

Neden üs alma sağdan sola? Matematiksel kural: $2^{3^2} = 2^9 = 512$ (doğru), $(2^3)^2 = 64$ (yanlış). Fortran ve Ruby bu geleneği takip eder. Visual Basic ise üs almayı sol associativity ile tanımlamıştır.

Dil	Sol Associativity	Sağ Associativity
Ruby	$*, /, +, -$	$**$
C / C++ / Java	$*, /, \%, \text{binary } +/ -$	$++, --, \text{unary } +/ -$
APL	—	Tüm operatörler
Visual Basic	$^$ dahil hepsi	—

2.3 Parentheses (Parantezler)

Parantezler, precedence ve associativity kurallarını **geçersiz kılmanın** tek yoludur. Parantezli bir alt expression, bitişik parantezsiz kısımlara göre her zaman daha yüksek önceliğe sahip olur.

```
1 (A + B) * C           // Toplama ONCE yapilir
2 (A + B) + (C + D)    // Overflow'u onlemek icin yeniden gruplama
```

“Parantez Her Şeyi Çözer” mi?

Teoride tüm öncelik kuralları kaldırılıp her şey parantezle yazılabilir — APL bu yaklaşımı seçmiştir. Ancak pratikte:

- Basit ifadeler bile aşırı parantezli hale gelir: $((a + (b * c)) - d)$
- Okunurluk **ciddi şekilde düşer**; matematiksel sezgiden kopulur.
- Yazım hataları (eksik parantez) daha sık yapılır.

Sonuç: Mevcut precedence kuralları matematik geleneğinden türetildiği için doğal ve sezgiseldir.

2.4 Özel Dil Yaklaşımları

Ruby — Operatörler Method Olarak

Ruby **saf nesne yönelimli (pure object-oriented)** bir dildir. Bu nedenle tüm aritmetik, ilişkisel ve atama operatörleri birer *method* olarak implement edilmiştir.

$a + b$ ifadesi aslında $a . + (b)$ çağrısıdır — a nesnesinin $+$ methodunu b parametresiyle çağırarak.

Sonuç: Bu operatörler kullanıcı tarafından **override** edilebilir. C++’da da operatörler fonksiyon çağrısı olarak implement edilmiştir (ancak method değil, standalone

fonksiyondur).

Lisp — Prefix Notation (Önek Gösterimi)

Lisp'te tüm aritmetik işlemler subprogram olarak gerçekleştirilir ve operatörler her zaman **prefix** biçiminde yazılır:

C ifadesi: $a + b * c$

Lisp karşılığı: $(+ a (* b c))$

Listede ilk eleman her zaman **fonksiyon adı**, diğerleri **parametreler**dir. Precedence belirsizliği yoktur; parantezler sırayı açıkça belirtir.

Conditional Expressions — Üçlü (Ternary) Operatör

C-tabanlı dillerde **conditional expression** üçlü operatör `?:` ile yazılır:

`koşul ? değer_eğer_doğru : değer_eğer_yanlış`

Örnek: `average = (count == 0) ? 0 : sum / count;`

`?` işareti *then* dalını, `:` işareti *else* dalını belirtir. Her iki dal zorunludur. Perl, JavaScript ve Ruby'de de desteklenir.

2.5 Operand Evaluation Order (Operand Değerlendirme Sırası)

Bir expression'daki operandların hangi sırayla değerlendirileceği çoğu dilde **tanımlanmamıştır** (**unspecified**). Bu derleyiciye optimizasyon özgürlüğü tanır. Ancak bir fonksiyon çağırısı **side effect** içeriyorsa, değerlendirme sırası sonucu doğrudan etkiler.

Tanım: Functional Side Effect (Fonksiyonel Yan Etki)

Bir fonksiyonun **kendi dönüş değeri dışında**, parametrelerinden birini veya global bir değişkeni **değiştirmesi** durumudur.

Matematiksel fonksiyonlarda side effect yoktur — çünkü matematikte “değişken” kavramı değiştirilemez bir simgeyi temsil eder. Aynı şekilde, pure functional programlama dillerinde fonksiyonlar yan etki üretmez.

```
1 // Side effect ornegi:
2 int a = 10;
3 // fun(&a): a'yi 20 yapar, 3 dondurur
4
5 b = a + fun(&a);    // Sonuc BELIRSIZ -- dile gore degisir!
6
7 // Soldan saga (Java garantisi):
8 //   Adim 1: a okunur -> 10
```

```

9 // Adım 2: fun(&a) cagirilir -> a=20, doner 3
10 // b = 10 + 3 = 13
11
12 // Sagdan sola (bazi C derleyicileri):
13 // Adım 1: fun(&a) cagirilir -> a=20, doner 3
14 // Adım 2: a okunur -> 20
15 // b = 20 + 3 = 23 <-- FARKLI SONUC!

```

C/C++ Belirsizliği

C ve C++ standartları operand değerlendirme sırasını **tanımsız bırakmıştır**. Bu, iki farklı C derleyicisinin (GCC vs MSVC) aynı koda farklı sonuç üretmesine yol açabilir: ciddi bir **taşınabilirlik (portability)** sorunudur.

Çözüm: Side effect içeren fonksiyon çağrılarını expression içine **gömmemek**, ayrı deyimler halinde yazmak en güvenli yaklaşımdır.

Java'nın Garantisi

Java dil tanımı, operandların **soldan sağa** değerlendirileceğini **garanti eder**. Aynı Java kodu tüm JVM implementasyonlarında aynı sonucu verir — platform bağımsızlığının temel taşlarından biri.

Trade-off: Bu garanti bazı derleyici optimizasyonlarını imkânsız kılar. Ancak Java'nın önceliği güvenilirlik ve taşınabilirlik olduğu için bu kabul edilebilir bir bedeldir.

2.6 Referential Transparency (Şeffaf Referans)

Tanım: Referential Transparency

Bir programda aynı değere sahip iki expression, birbirinin yerine programın çalışmasını değiştirmeden kullanılabiliriyorsa, program **referentially transparent** özelliğindedir.

Teknik olarak: Bir fonksiyonun dönüş değeri **yalnızca parametrelerine** bağlıysa (global değişken okuyup değiştirmiyorsa), o fonksiyon referentially transparent'tır.

```

1 // fun() side effect icermiyorsa: result1 == result2
2 result1 = (fun(a) + b) / (fun(a) - c);
3 temp    = fun(a);
4 result2 = (temp + b) / (temp - c);
5
6 // Eger fun(), b veya c'yi degistiriyorsa:
7 // fun(a) ilk cagrida farkli, ikinci cagrida farkli deger
   dondurur
8 // result1 != result2 --> Referential transparency BOZULMUS

```


Referential Transparency'nin Avantajları

- **Anlaşılabilirlik:** Fonksiyon bağlamdan bağımsız; yalnızca parametrelerine bakarak ne yaptığı anlaşılır.
- **Test kolaylığı:** Aynı girdiler her zaman aynı çıktıyı üretir; unit test yazmak çok kolaydır.
- **Paralleleştirme:** Yan etki olmadığından fonksiyonlar güvenle paralel çalıştırılabilir.
- **Derleyici optimizasyonu:** Aynı argümanlarla iki çağrı varsa, derleyici ikincisini birincinin sonucuyla ikame edebilir.

Pure functional dillerde (Haskell, ML, F#) referential transparency doğal olarak sağlanır.

3 Overloaded Operators

Tanım: Operator Overloading

Operator overloading, bir operatör sembolünün **birden fazla anlam** için kullanılmasıdır. Bu kullanım, okunurluk ve güvenilirlik zedelenmediği sürece genel olarak kabul edilebilir sayılır.

Neden overloading gereklidir? Gerçek dünyada “toplama” işlemi farklı tipler için farklı anlam taşır: tam sayı toplamı, kayan nokta toplamı, matris toplamı, vektör toplamı... Tek bir + sembolüyle tüm bu durumların ifade edilebilmesi doğal ve sezgiseldir.

Operatör	Kullanım 1	Kullanım 2	Kullanım 3
(C++)	Integer toplama	Float toplama	String concat (Java)
	Binary çıkarma	Unary negatif ($-x$)	—
	Integer/Float çarpma	User-defined (matris)	—
	Binary: Bitwise AND	Unary: Address-of ($\&x$)	—
	Bitwise left shift	Stream output (C++)	—

3.1 Tehlikeli Overloading: C++’da & Operatörü

Tehlike: & Operatörünün İki Anlamı (C++)

C++’da $\&$ sembolü tamamen **ilişkisiz** iki operasyon için kullanılır:

- **Binary operatör:** $x \ \& \ y \rightarrow$ Bitwise AND
- **Unary operatör:** $\&x \rightarrow$ x’in bellek adresi

Sorun 1: Birbirinden tamamen farklı işlemler için aynı sembol okunurluğu zedeler.

Sorun 2: Binary AND yazılırken ilk operand unutulursa, derleyici bunu unary address-of olarak yorumlar — **hata mesajı vermez!**

```

1 int x, y, z;
2 z = x & y;    // Binary: Bitwise AND -- dogru kullanım
3
4 // Programci ilk operandi unuttu -- AMA DERLEME HATASI YOK:
5 z = & y;      // Unary: &y = y'nin adresi
6              // Derleyici suskundur; anlamsiz ama gecerli kod.
```

3.2 User-Defined Operator Overloading

C++, C# ve F# programcılarının kendi operatör tanımlarını yapmasına izin verir.

```

1 // Matris sinifi icin + ve * tanimlanmissa:
2 A * B + C * D
```

```
3
4 // Aksi halde soyle yazmak gerekirdi:
5 MatrixAdd(MatrixMult(A, B), MatrixMult(C, D))
```

User-Defined Overloading'in Riskleri

- **Anlam kaybı:** Hiçbir şey `+`'ı çarpma olarak tanımlamayı engellemez!
- **Gizli karmaşıklık:** Bir operatör gördüğünüzde, anlamını bulmak için operand tiplerini ve tüm overload tanımlarını aramanız gerekir — tanımlar başka dosyalarda olabilir.
- **Ekip uyumsuzluğu:** Farklı modüller aynı operatörü farklı biçimde overload ederse, sistem entegrasyonu sırasında çelişkiler ortaya çıkar.

Java bu riski sıfırlamak için user-defined overloading'i hiç eklemedi. **C#** ise ekleyerek programcıya sorumluluk bıraktı.

4 Type Conversions (Tip Dönüşümleri)

Bir expression'da farklı tiplere sahip operandlar bir araya geldiğinde, derleyicinin bu tip uyumsuzluğunu çözmesi gerekir. Çözüm iki biçimde gerçekleşebilir: **implicit (örtük)** dönüşüm (coercion) veya **explicit (açık)** dönüşüm (cast).

4.1 Widening ve Narrowing Conversions

Tanım: Widening Conversion (Genişletme)

Orijinal tipin **tüm değerlerini** (en azından yaklaşık olarak) temsil edebilen, daha büyük/kapsamlı bir tipe yapılan dönüşümdür. Genellikle **güvenlidir**.

Örnek: `int` → `float`.

DİKKAT: Widening her zaman mükemmel değildir! Java'da 32-bit `int` → 32-bit `float` dönüşümünde hassasiyet kaybı yaşanabilir: `int` 9+ basamak taşıyabilirken, `float` yalnızca ≈ 7 basamak saklayabilir.

Tanım: Narrowing Conversion (Daraltma)

Orijinal tipin tüm değerlerini **karşılamayan**, daha küçük/kısıtlı bir tipe yapılan dönüşümdür. **Veri kaybı veya hata riski** taşır.

Örnek: `double` → `float`.

Daha aşırı: `(int)1.3e25` — Bu değer `int` aralığına sığmaz; sonuç orijinal değerle **hiçbir ilgisi olmayan** rastgele bir sayıdır.

Java'da genişletme hiyerarşisi:

```
1 byte -> short -> int -> long -> float -> double
2           char -> int    (char ayrı bir widening yoluna sahip
3                       )
4 // Tüm bu dönüşümler otomatik (implicit) gerçekleşir.
5 // Ters (örnek: double -> int) için açık cast gereklidir.
```

4.2 Coercion in Expressions

Tanım: Coercion

Coercion, derleyici veya çalışma zamanı ortamı tarafından **otomatik olarak** gerçekleştirilen örtük (implicit) tip dönüşümüdür. Programcı herhangi bir şey yazmaz; derleyici gerekli dönüşüm kodunu üretir.

Programcının açıkça belirttiği dönüşümler ise **cast** veya **explicit conversion** olarak adlandırılır.

Coercion ve Hata Tespiti Sorunu

Geniş kapsamlı coercion kuralları hata tespitini zayıflatır:

```
1 int a;
2 float b, c, d;
3 d = b * a; // 'a' yerine 'c' yazılması gerekirdi
4           // Derleyici hata VERMEZ -- a, float'a coerce edilir
```

Mixed-mode expression yasak olsaydı (Ada, ML gibi) → **Derleme zamanı tip hatası!** → Hata erken yakalanırdı.

Bu nedenle **F#, Ada, ML** mixed-mode expression'lara izin vermez.

4.3 Explicit Type Conversion (Cast)

```
1 // C / C++ / Java / C#:
2 (int)angle           // "angle" degerini int olarak yorumla
3 (float)(a + b)       // Toplam once hesaplanir, sonra float'a
                       donusturulur
4
5 // ML / F# (fonksiyon cagri seklinde):
6 float(sum)
7 int(3.7)             // --> 3 (kesir atilir)
```

Dil	Mixed-Mode İzni	Coercion	Güvenlik
C, C++, Perl	Geniş	Çok geniş	Düşük
Java, C#	Yalnızca widening	Kısıtlı	Yüksek
Ada, ML, F#	Hayır	Yok	En Yüksek
Python, Ruby	N/A	—	—

5 Relational and Boolean Expressions

5.1 Relational Expressions

Tanım: Relational Expression

Bir **relational operator**, iki operandın değerini karşılaştırır ve bir **Boolean** sonuç üretir. Relational expression iki operand ve bir relational operatörden oluşur.

Öncelik: Relational operatörler, aritmetik operatörlerden her zaman daha düşük önceliğe sahiptir. Böylece $a + 1 > 2 * b$ ifadesinde önce aritmetik işlemler yapılır.

Karşılaştırma	C/Java	Fortran 95+	ML/F#	Ruby
Eşit	<code>==</code>	<code>.EQ. / =</code>	<code>=</code>	<code>==</code>
Eşit Değil	<code>!=</code>	<code>.NE. / <></code>	<code><></code>	<code>!=</code>
Küçük	<code><</code>	<code>.LT. / <</code>	<code><</code>	<code><</code>
Büyük	<code>></code>	<code>.GT. / ></code>	<code>></code>	<code>></code>
Küçük Eşit	<code><=</code>	<code>.LE. / <=</code>	<code><=</code>	<code><=</code>
Büyük Eşit	<code>>=</code>	<code>.GE. / >=</code>	<code>>=</code>	<code>>=</code>

Tarihsel Not: Fortran'ın .EQ. Sözdizimi

Fortran I (1957) tasarımcıları, relational operatörler için `<` ve `>` sembollerini kullanamadı. Çünkü o dönemin **kart delgicilerinde (card punches)** bu semboller yoktu! Bu nedenle İngilizce kısaltmalar tercih edildi: `.EQ.`, `.NE.`, `.LT.`, `.GT.`, vb. Fortran 95 ile modern semboller de kabul edildi.

JavaScript/PHP: `==` vs `===`

- `==` (loose equality): Operandlar **coerce edilir**. `"7" == 7` → `true` (string sayıya çekildi)
- `===` (strict equality): Coerce edilmez; tip ve değer aynı olmalı. `"7" === 7` → `false`

Ruby'de `==` coercion yaparken, `eql?` yapmaz. Güvenilir kod için `===` kullanılması önerilir.

5.2 Boolean Expressions

Operasyon	C-tabanlı	Ada	Python	Ruby
AND	&&	and	and	&& / and
OR		or	or	/ or
NOT	!	not	not	! / not
Bitwise AND	&	---	&	&
Bitwise OR		---		

C-tabanlı dillerde tam operatör öncelik sırası (en yüksekte en düşüğe):

Sıra	Operatörler
1 (En Yüksek)	postfix ++, --
2	unary +, -, prefix ++, --, !
3	*, /, %
4	binary +, -
5	<, >, <=, >=
6	==, !=
7	&& (AND)
8 (En Düşük)	(OR)

C'de Boolean: Tip Yok! (C99 öncesi)

C99 öncesinde C'nin ayrı bir Boolean tipi yoktu. Sayısal değerler kullanılırdı: **0 = false**, **sıfır olmayan her şey = true**.

Tuhaf sonuç: `a > b > c` ifadesi **geçerlidir** — ama `b` ile `c`'yi *karşılaştırmaz*! Önce `a > b` (0 ya da 1) hesaplanır, sonra bu değer `c` ile karşılaştırılır.

C99 çözümü: `<stdbool.h>` ile `bool`, `true`, `false` kullanılabilir hale geldi.

6 Short-Circuit Evaluation

Tanım: Short-Circuit Evaluation

Short-circuit evaluation, bir Boolean expression'ın tüm operand ve operatörleri değerlendirilmeden, **sonucun ilk operanddan belirlendiği durumlarda değerlendirmeyi erken sonlandırmaktır.**

- AND: İlk operand `FALSE` ise $\rightarrow$ tüm expression `FALSE`; ikinci operand değerlendirilmez.
- OR: İlk operand `TRUE` ise $\rightarrow$ tüm expression `TRUE`; ikinci operand değerlendirilmez.

İfade	1. Operand	2. Operand?	Sonuç
	<code>FALSE</code>	ATLANIR	<code>FALSE</code>
	<code>TRUE</code>	Değerlendirilir	B'ye göre
	<code>TRUE</code>	ATLANIR	<code>TRUE</code>
	<code>FALSE</code>	Değerlendirilir	B'ye göre

6.1 Pratik Kullanım: Dizi Sınır Kontrolü

```

1 index = 0;
2 while ((index < listlen) && (list[index] != key))
3     index = index + 1;
4
5 // Short-circuit OLMADAN:
6 // index == listlen olduğunda:
7 // 1. (index < listlen) --> FALSE
8 // 2. Ama list[index] DA değerlendirilir! --> list[listlen] -->
   COKME
9 //
10 // Short-circuit İLE (Java, C++, Python...):
11 // 1. (index < listlen) --> FALSE
12 // 2. İkinci kısım ATLANIR --> güvenli

```

6.2 Short-Circuit + Side Effect = Tehlikeli Kombinasyon

Gizli Hata Tuzağı

```

// Java:
(a > b) || ((b++) / 3)
// a > b TRUE ise: (b++) CALISTIRILMAZ --> b degismez!
// Eger kodun geri kalanı b'nin her seferinde
// artacağını varsayıyorsa --> SESSİZ HATA

```


Bu tür hatalar yalnızca belirli koşullarda ortaya çıkar ve bulmak çok güçtür.

Dil	&& / Short-Circuit?	Notlar
C, C++	&& ve evet	& ve hayır (bitwise)
Java, C#	&& ve evet	Boolean için de & ve var
Python	Tümü (and, or)	Yalnızca and/or var
Ruby, Perl, ML, F#	Tümü	
Ada	and/or hayır	and then / or else short-circuit

Ada'nın Özel Çözümü

Ada programcıya seçim hakkı tanır: `and` ve `or` her iki operandı da değerlendirir (tam değerlendirme). `and then` ve `or else` ise short-circuit uygular. Bu yaklaşım daha açık ve kontrol edilebilir bir tasarım sunar.

7 Assignment Statements

Assignment statement (atama deyimi), imperative programlamanın merkezinde yer alır. Değişkenlere yeni değer bağlayan ve programın durumunu güncelleyen bu deyim, tarihsel süreçte pek çok farklı biçimde evrilmiştir.

7.1 Simple Assignment (Basit Atama)

Modern dillerin büyük çoğunluğu atama için `=` sembolünü kullanır. Ancak bu, eşitlik operatörü `==` ile karışıklığa yol açar. **ALGOL 60** bu sorunu çözmek için `:=` sözdizimini icat etti. **Ada** da bu geleneği sürdürmektedir.

```

1 // C / C++ / Java / Python:
2 a = b + c;           // = atama
3 if (a == b) ...     // == esitlik testi
4
5 -- Ada / ALGOL:
6 a := b + c;         -- := atama
7 if a = b then       -- = esitlik testi

```

7.2 Assignment Çeşitleri

Tür	Sözdizimi	Açıklama
Simple	<code>a = b + c</code>	Temel atama
Conditional Target	<code>(\$flag?\$a:\$b) = 0</code>	Koşullu hedef (Perl)
Compound	<code>sum += value</code>	<code>sum = sum + value</code> kısaltması
Unary	<code>count++, --x</code>	1 artır/azalt
As Expression	<code>while((ch = getchar()) != EOF)</code>	Atama değer üretir (C)
Multiple	<code>(\$a, \$b) = (\$b, \$a)</code>	Çoklu kaynak-hedef (Perl/Ruby)

7.3 Compound Assignment Operators

Tanım: Compound Assignment

Compound assignment operator, hedef değişkenin aynı zamanda expression'ın ilk operandı olduğu yaygın kalıbı kısaltmak için kullanılır. Genel form: `hedef op = ifade` $\equiv$ `hedef = hedef op ifade`

Tarih: ALGOL 68'de tanıtıldı. C'de biraz farklı formla benimsendi. Bugün C, C++, Java, C#, Python, JavaScript, Ruby, Perl'de desteklenir.

```

1 sum += value;           // sum = sum + value
2 total -= discount;     // total = total - discount

```

```

3 count *= 2;           // count = count * 2
4 x      /= 4;          // x = x / 4
5 r      %= 7;          // r = r % 7

```

7.4 Unary Assignment Operators — ++ ve --

C-tabanlı diller, Perl ve JavaScript'te ++ ve -- operatörleri aritmetik ve atamayı tek adımda birleştirir. Hem **prefix** (ön ek) hem de **postfix** (son ek) olarak kullanılabilir ve bu iki kullanımın anlamı birbirinden farklıdır!

Kullanım	Anlamı	Eşdeğer Kod
	Önce artır, sonra ata	count =count+1; sum =count;
	Önce ata, sonra artır	sum =count; count =count+1;
	Sadece artır	count = count + 1;
	Önce artır, sonra negat	-(count++)

Tarihsel Not: ++ Operatörü ve PDP-11

++ ve -- operatörleri, C'nin temel aldığı **B dilinden (1969)** miras alınmıştır. Yaygın bir yanlışlığı: "PDP-11 mimarisinden esinlenilerek eklendi." Aslında **tam tersi**: Operatörler önce B dilinde vardı; PDP-11'in autoincrement/autodecrement adresleme modları sonradan bu dil özelliğini doğrudan destekledi.

7.5 Assignment as an Expression

C-tabanlı dillerde, Perl ve JavaScript'te atama deyimi bir **değer üretir**: atanan değer. Bu sayede atama, başka bir expression'ın operandı olarak kullanılabilir.

Klasik Kullanım: while ile getchar()

```

while ((ch = getchar()) != EOF) {
    // ch'ye karakter okunur ve EOF ile karsilastirilir
}
// Parantez ZORUNLU! Cunku = operatorunun onceligi
// != operatöründen DUSUKTUR.

```

= vs == Karıştırma Tehlikesi

```

if (x = y) ... // Hata degil! y, x'e atanir, sonra test edilir
if (x == y) ... // Dogru: esitlik testi

```

Bu hata derleyici tarafından yakalanmaz. **Java ve C#** bu hatayı imkânsız kılmak için **if** koşulunda yalnızca **boolean expression** kabul eder.

7.6 Multiple Assignments (Perl, Ruby)

```
1 # Perl: coklu atama
2 ($first, $second, $third) = (20, 40, 60);
3
4 # Gecici degisken gerektirmeyen swap:
5 ($first, $second) = ($second, $first);
6 # Sag taraf once tamamen degerlendirilir, sonra sol tarafa atanir.
```

7.7 Assignment in Functional Programming

Fonksiyonel Dillerde “Atama” Kavramı

Pure functional dillerde tüm identifier’lar yalnızca **değer isimleridir** — değiştirilebilir değişken değil. Değer bir kez bağlandıktan (bind) sonra değişmez.

- **ML’de** `val` bildirimi bir isme değer bağlar. Aynı isim tekrar bildirilirse yeni bir bağlama oluşur; eskisi gizlenir (shadow).
- **F#’de** `let` bildirimi benzerdir ama yeni bir kapsam (scope) oluşturur.

8 Mixed-Mode Assignment

Tanım: Mixed-Mode Assignment

Atama deyiminin **sol tarafındaki değişkenin tipi** ile **sağ taraftaki expression'ın tipi** farklıysa, bu bir **mixed-mode assignment** olarak adlandırılır.

Tasarım sorusu: Dil, bu durumda **otomatik dönüşüme (coercion)** izin vermeli mi, yoksa **açık cast** zorunlu olmalı mı?

Dil	Kural	Güvenlik
C, C++, Perl	Geniş coercion; çoğu tip karışımına izin	Düşük
Java, C#	Yalnızca widening coercion	Yüksek
Ada, ML, F#	Hiç mixed-mode yok; açık cast zorunlu	En Yüksek
Python, Ruby	Tipler nesnelere ait; kavram yok	—

```

1 // Java Mixed-Mode Assignment:
2 int i = 5;
3 float f = i;           // İZINLI -- widening (int --> float)
4
5 double d = 3.14;
6 int n = d;             // HATA -- narrowing, cast gerekli
7 int m = (int) d;       // İZINLI -- açık cast ile (3)
8
9 // Java'nın özel kuralı (literal için istisna):
10 byte b = 100;          // İZINLI -- 100, byte aralığında (0-127)
11 byte c = 200;          // HATA -- 200, byte aralığını aşar

```

Java/C# Yaklaşımının Avantajı

Java ve C#, yalnızca widening mixed-mode assignment'a izin vererek tip hata tespitinin yarısını kurtarır. Bu, C/C++'a göre çok daha güvenilir bir yaklaşımdır; programcıların dikkatsiz tip karıştırma hatalarını derleme zamanında yakalar.

Bölüm Özeti: Tüm Kavramlar Bir Bakışta

Kavram	Tanım	Anahtar Diller
Operator Precedence	Farklı öncelikteki ops değerlendirme sırası	Tüm diller (APL hariç)
Associativity	Eşit öncelikteki ops yönü	Sol: çoğu; Sağ: ** ve unary
Side Effect	Fonksiyonun parametre/global değişirmesi	Tüm imperative diller
Ref. Transparency	Aynı değer → her yerde kullanılabilir	ML, F#, Haskell
Operator Overloading	Sembol birden fazla anlam taşır	C++, C#, Python, Ruby
Widening	Büyük tipe güvenli dönüşüm	Java, C# (otomatik)
Narrowing	Küçük tipe riskli dönüşüm	Cast ile tüm diller
Coercion	Derleyici otomatik tip dönüşümü	C, C++, Java (kısıtlı)
Short-Circuit	Boolean: erken sonuç → 2. op atlanır	Java, C, Python, Ruby
Compound Assign.	<code>op</code> = kısaltma formu	C-tabanlı, Python, Ruby
Unary ++/--	Tek operandlı artır/azalt ataması	C-tabanlı, Perl, JS
Multiple Assign.	Çoklu kaynak-hedef atama	Perl, Ruby, Python

9 Review Questions — Soru Çözümleri

S1. Define operator precedence and operator associativity.

Cevap

Operator precedence: Farklı öncelik seviyelerindeki operatörlerin bir expression içinde hangi sırada değerlendirileceğini belirleyen kuraldır. Örneğin $a + b * c$ 'de $*$, $+$ 'dan yüksek öncelikli olduğu için çarpma önce yapılır: $a + (b * c)$.

Operator associativity: Aynı öncelik düzeyindeki ve birbirine bitişik iki operatörün hangi sırayla değerlendirileceğini belirler. Sol associativity'de soldaki önce: $a - b + c \rightarrow (a - b) + c$. Sağ associativity'de sağdaki önce: $A ** B ** C \rightarrow A ** (B ** C)$.

S2. What is a ternary operator?

Cevap

Ternary operator (üçlü operatör), tam olarak **üç operand** alan bir operatördür. C-tabanlı dillerdeki koşullu operatör `?:` bunun klasik örneğidir:

```
koşul ? değer_doğruysa : değer_yanlışısa
```

Örnek: `average = (count == 0) ? 0 : sum / count;`

S3. What is a prefix operator?

Cevap

Prefix operator, operandından **önce** gelen operatördür. C'deki `++x` ve `-x` prefix operatörlere örnektir. Lisp'te tüm operatörler prefix'tir: `(+ a b)`. Perl'de de bazı operatörler prefix biçimindedir.

S4. What operator usually has right associativity?

Cevap

Exponentiation (üs alma) operatörü (`**` ya da `^`) genellikle **sağ associativity**'e sahiptir. Fortran ve Ruby'de $A ** B ** C \rightarrow A ** (B ** C)$ olarak değerlendirilir. Bu, matematiksel geleneğe uygundur: $2^{3^2} = 2^9 = 512$.

C-tabanlı dillerde `++`, `--` ve unary `-` gibi unary operatörler de sağ associativity'e sahiptir.

İstisna: Visual Basic'te üs alma operatörü sol associativity'e sahiptir.

S5. What is a nonassociative operator?

Cevap

Nonassociative operator, aynı operatörün ard arda kullanılmasına izin vermeyen operatördür. Bazı dillerde $a < b < c$ yazımına izin verilmez; bu ifade ya derleme

hatası verir ya da özel sözdizimi gerektirir. Bu kısıtlamanın amacı matematiksel olmayan yorumların önüne geçmektir. C’de $a > b > c$ geçerlidir ama beklenen karşılaştırmayı yapmaz!

S6. What associativity rules are used by APL?

Cevap

APL’de **tek bir öncelik düzeyi** vardır; tüm operatörler eşit öncelikte kabul edilir. Değerlendirme sırası tamamen **sağdan sola (right-to-left) associativity** ile belirlenir. $A * B + C \rightarrow A * (B + C)$ olarak değerlendirilir. $A = 3, B = 4, C = 5$ ise sonuç 27 olur.

S7. What is the difference between the way operators are implemented in C++ and Ruby?

Cevap

C++: Operatörler derleyici tarafından *fonksiyon çağırısı* olarak implement edilir. Kullanıcı `operator+()` şeklinde tanımlayabilir.

Ruby: Operatörler tam anlamıyla **method**’tur. $a + b, a.(+)(b)$ çağırısıyla eşdeğerdir. Bu sayede operatörler doğrudan **override** edilebilir. Her ikisi de user-defined overloading destekler; ancak Ruby’nin yaklaşımı daha bütünleşik (nesne yönelimli) bir tasarım sunar.

S8. Define functional side effect.

Cevap

Functional side effect, bir fonksiyonun kendi dönüş değeri dışında, **parametrelerinden birini veya global bir değişkeni değiştirmesi** durumudur. Side effect içeren fonksiyonlar, operand değerlendirme sırasına bağımlı sonuçlar üretir; bu da aynı kodun farklı derleyicilerde farklı sonuç vermesine yol açabilir.

S9. What is a coercion?

Cevap

Coercion, derleyici veya çalışma zamanı ortamı tarafından **otomatik olarak** gerçekleştirilen örtük (implicit) tip dönüşümüdür. Programcı herhangi bir şey yazmaz. Örneğin Java’da `int` değer, `float` beklenen bir yerde otomatik olarak dönüştürülür. Coercion, tip sisteminin bir güvenlik faydasını (hata tespiti) zayıflatır.

S10. What is a conditional expression?

Cevap

Conditional expression, bir koşulun sonucuna göre iki farklı değer üreten, ternary

operatörünü kullanan expression'dır.

`koşul ? değer1 : değer2`

Örnek: `max = (a > b) ? a : b;` — if-else bloğunu tek satırda ifade eder. Her iki dal zorunludur.

S11. What is an overloaded operator?

Cevap

Overloaded operator, birden fazla anlam için kullanılan operatör sembolüdür. Örneğin `+` operatörü hem tam sayı hem kayan nokta toplamı hem de (Java'da) string birleştirmesi için kullanılır. İki tür overloading vardır: (1) **Built-in**: Dilin kendisi tarafından tanımlanmış çoklu anlam. (2) **User-defined**: Programcının tanımladığı çoklu anlam (C++, C#, F#, Python, Ruby).

S12. Define narrowing and widening conversions.

Cevap

Widening conversion: Orijinal tipin tüm değerlerini temsil edebilen daha büyük/kapsamlı bir tipe dönüşüm. Genellikle **güvenlidir**. Örnek: `int` → `float`. Ancak 32-bit `int` → 32-bit `float` dönüşümünde hassasiyet kaybı olabilir.

Narrowing conversion: Orijinal tipin tüm değerlerini karşılamayan daha küçük tipe dönüşüm. **Veri kaybı riski** taşır. Örnek: `double` → `float`. Java'da yalnızca widening mixed-mode assignment'a izin verilir.

S13. In JavaScript, what is the difference between `==` and `===`?

Cevap

`==` (loose equality): Operandlar **coerce edilir**. `"7" == 7` → `true` (string sayıya çekildi).

`===` (strict equality): Operandlar **coerce edilmez**; tip ve değer her ikisi de aynı olmalı. `"7" === 7` → `false`.

Güvenilir kod için `===` kullanılması önerilir. PHP'de de aynı ayırım geçerlidir. Ruby'de `==` coercion yaparken `eq?` yapmaz.

S14. What is a mixed-mode expression?

Cevap

Mixed-mode expression, farklı tiplere sahip operandları içeren aritmetik expression'dır. Örneğin bir `int` ile bir `float`'ın aynı expression'da kullanılması. Derleyici, uyumsuz tipleri otomatik coerce eder. Ada ve ML bu tür expression'lara izin vermez; açık tip dönüşümü zorunludur.

S15. What is referential transparency?**Cevap**

Referential transparency, bir programda aynı değere sahip iki expression'ın birbirinin yerine, programın çalışmasını değiştirmeden kullanılabilmesidir. Bir fonksiyonun dönüş değeri yalnızca parametrelerine bağlıysa (global değişken okuyup değiştirmiyorsa), o fonksiyon referentially transparent'tır. Pure functional dillerde bu özellik doğal olarak sağlanır.

S16. What are the advantages of referential transparency?**Cevap**

- **Anlaşılabilirlik:** Fonksiyon bağlamdan bağımsız; sadece parametrelerine bakarak ne yaptığı anlaşılır.
- **Test kolaylığı:** Aynı girdiler her zaman aynı çıktıyı üretir; unit test yazmak çok kolaydır.
- **Parallelleştirme:** Yan etki olmadığından fonksiyonlar güvenle paralel çalıştırılabilir.
- **Derleyici optimizasyonu:** Aynı argümanlarla iki çağrı varsa, derleyici ikincisini birincinin sonucuyla ikame edebilir.
- **Matematiksel doğruluk:** Fonksiyonlar matematiksel tanımlarıyla örtüşür; formal doğrulama kolaylaşır.

S17. How does operand evaluation order interact with functional side effects?**Cevap**

Bir fonksiyon çağrısı expression'daki bir değişkeni (parametre veya global) değiştiriyorsa, operandların hangi sırada değerlendirileceği **nihai sonucu etkiler**. Örnek: `b = a + fun(&a);` — `fun, a`'yı değiştiriyorsa: `a` önce okunursa farklı, `fun` önce çağrılırsa farklı sonuç.

Çözüm yolları: (1) Side effect'leri yasaklamak (Ada, ML). (2) Belirli bir sıra garanti etmek (Java: soldan sağa).

S18. What is short-circuit evaluation?**Cevap**

Short-circuit evaluation, bir Boolean expression'ın tüm operand ve operatörleri değerlendirilmeden, **sonucun ilk operandtan belirlendiği durumda değerlendirmeyi erken sonlandırmaktır**. AND: ilk operand FALSE ise ikinci atlanır. OR: ilk operand TRUE ise ikinci atlanır. Null pointer kontrolü ve dizi sınır kontrolü gibi durumlarda çalışma zamanı hatalarını önler.

S19. Name a language that always does short-circuit evaluation. Name one that never does it.

Cevap

Her zaman short-circuit yapan: Java (`&&` ve `||`), Python, Ruby, Perl, ML, F# — bu dillerde tüm logical operatörler short-circuit'tir.

Short-circuit yapmayan (varsayılan): Ada'da `and` ve `or` operatörleri short-circuit değildir. Short-circuit için `and then` ve `or else` kullanılmalıdır. C'deki `&` ve `|` bit operatörleri de short-circuit değildir.

S20. How does C support relational and Boolean expressions?

Cevap

C99 öncesinde C'nin ayrı bir **Boolean tipi yoktu**. Sayısal değerler kullanılırdı: `0 = false`, **sıfır olmayan her şey = true**. Relational expression'lar `int` değer üretir (0 ya da 1). Herhangi bir sayısal expression Boolean operand olarak kullanılabilirdi — bu hata tespitini azaltırdı. `a > b > c` ifadesi geçerlidir ama b ile c'yi karşılaştırmaz!

C99 çözümü: `<stdbool.h>` ile `bool`, `true`, `false` eklendi.

S21. What is the purpose of a compound assignment operator?

Cevap

Compound assignment operator, hedef değişkenin aynı zamanda expression'ın ilk operandı olduğu yaygın kalıbı kısaltmak için kullanılır. Örneğin `sum = sum + value` yerine `sum += value` yazılır. (1) Okunurluk artar. (2) Uzun değişken isimlerinde tekrar yazmaktan kaçınılır. (3) Yazım hatası riski azalır. ALGOL 68 kökenlidir.

S22. What is the associativity of C's unary arithmetic operators?

Cevap

C'deki unary aritmetik operatörler (`++`, `--`, unary `-`, unary `+`) **sağdan sola (right-to-left) associativity**'e sahiptir. Örnek: `-count++` ifadesi önce `count`'u artırır (`count++`), ardından negasyonu uygular: `-(count++)` şeklinde değerlendirilir. **Değil:** `(-count)++`.

S23. What is one possible disadvantage of treating the assignment operator as an arithmetic operator?

Cevap

En büyük dezavantaj, atama (`=`) ile eşitlik testi (`==`)'in karıştırılmasının **derleyici tarafından yakalanamamasıdır**. `if (x = y) ...` — Hata değil! `y`, `x`'e atanır, sonra `x`'in değeri test edilir. Ayrıca assignment'ı expression olarak kullanmak kodu karmaşılaştırır ve beklenmedik side effect'lere yol açabilir.

S24. What two languages include multiple assignments?**Cevap**

Perl ve **Ruby** çoklu hedef-kaynaklı atama deyimlerini destekler. Perl örneği: `($first, $second, $third) = (20, 40, 60);`
Değer değiştirme (swap): `($first, $second) = ($second, $first);` — geçici değişken gerektirmez! Python da çoklu hedefli atamayı destekler: `sum = count = 0.`

S25. What mixed-mode assignments are allowed in Java?**Cevap**

Java, yalnızca **widening (genişletme)** coercion gerektiren mixed-mode assignment'lara izin verir.

İzinli: `float x = 5;` — `int` → `float` (widening).

Yasak: `int y = 3.5f;` — `float` → `int` (narrowing); açık cast gerekli.

İstisna: `byte b = 100;` izinlidir — 100, byte aralığında (0–127); değer kaybı yok.

S26. What mixed-mode assignments are allowed in ML?**Cevap**

ML'de mixed-mode assignment **hiçbir şekilde izin verilmez**. Değer bağlamak için kullanılan `val` bildirimi, sol ve sağ tarafın tiplerinin birebir eşleşmesini zorunlu kılar. Farklı tipler için açıkça **cast (explicit conversion)** kullanmak zorunludur. Not: ML'de “universal” tipler (integer ve real literaller) vardır; bunlar uygun tipler için polimorfik sayılır — ancak bu tam anlamıyla coercion değildir.

S27. What is a cast?**Cevap**

Cast (açık tip dönüşümü), programcı tarafından açıkça istenen explicit tip dönüşümüdür; coercion'ın aksine otomatik değildir.

C / C++ / Java / C#: `(int) angle`

ML / F#: `float (sum)` — fonksiyon çağrısı biçiminde.

Widening ve narrowing olarak ikiye ayrılır. Narrowing cast'ler veri kaybına yol açabilir ve bazı dillerde uyarı mesajı üretir.

10 Problem Set — Soru Çözümleri

P1. When might you want the compiler to ignore type differences in an expression?

Cevap

Lehte argümanlar:

- **Pratiklik:** $3 + 1.5$ yazmak $3.0 + 1.5$ 'den daha doğal; her yerde açık cast zahmetlidir.
- **Hız:** Özellikle prototip geliştirmede tip uyumluluğunu sürekli düşünmek yaratıcılığı sekteye uğratır.
- **Genel amaçlı işlemler:** Hem tam sayılara hem kayan noktalılara uygulanabilen tek bir fonksiyon istenebilir.

Unutmayın: Coercion hata tespitini zayıflatır. Güvenlik odaklı projelerde coercion mümkün olduğunca kısıtlanmalıdır.

P2. State your own arguments for and against allowing mixed-mode arithmetic expressions.

Cevap

Lehine:

- Kod daha az verbose; her yerde açık cast gerekmez.
- Matematiksel düşünce biçimiyle uyumlu ($3 + 1.5 = 4.5$).
- Hızlı prototipleme kolaylaştırır.

Aleyhine:

- Yanlışlıkla yanlış tip kullanılması derleyici hatası vermez.
- Coercion'ın hangi yönde yapılacağı her zaman sezgisel değildir.
- Açık dönüşüm daha iyi bir yazım alışkanlığı oluşturur.

Denge: Java'nın yaklaşımı (yalnızca widening) iyi bir uzlaşma noktası sunar.

P3. Do you think the elimination of overloaded operators in your favorite language would be beneficial?

Cevap (Örnek: Java)

Lehine (kaldırılması iyi olur):

- Operatör anlamını bulmak için overload tanımlarını aramak gerekmez — okunurluk artar.
- Farklı modüllerin çelişkili overload tanımlarından kaynaklanan sorunlar yaşanmaz.

Aleyhine:

- Matris, karmaşık sayı gibi matematiksel tiplerde `+` ve `*` kullanmak daha doğal kod sağlar.

Java'nın tercihi iyi bir denge noktasıdır.

P4. Would it be a good idea to eliminate all operator precedence rules and require parentheses?

Cevap

Hayır, iyi bir fikir değildir.

Avantajları: Tek kural; herhangi bir operatörün önceliğini ezberlemek gerekmez. APL bu yaklaşımı benimsemiştir.

Dezavantajları:

- Basit ifadeler bile aşırı parantezli hale gelir.
- Okunurluk ciddi şekilde düşer; matematiksel sezgiden kopulur.
- Parantez eksikliği hataları daha sık yapılır.

Sonuç: Mevcut precedence kuralları matematiksel gelenekten türetildiği için doğal ve sezgiseldir.

P5. Should C's assigning operations (e.g., `+=`) be included in other languages?

Cevap

Evet, dahil edilmelidir.

- `x = x + y` yerine `x += y` daha kısadır ve okunurluk artar.
- Uzun değişken isimlerinde tekrar yazmaktan kaçınılır.
- Pek çok modern dil (Python, JavaScript, Ruby, Kotlin) zaten benimsemiştir.
- Derleyici optimizasyonuna katkı sağlayabilir.

P6. Should C's single-operand assignment forms (e.g., `++count`) be included in other languages?

Cevap

Tartışmalıdır.

Lehine: Döngü sayaçlarında `i++` kısadır ve yaygındır.

Aleyhine:

- Prefix–postfix arasındaki ince anlam farkı sürpriz hatalara yol açabilir.
- Python kasıtlı olarak dahil etmemiştir; `count + = 1` daha açık ve güvenlidir.
- Expression içinde kullanıldığında side effect üretir.

P7. Describe a situation in which the add operator would not be commutative.

Cevap

Java'da + operatörü string birleştirme için de kullanılır ve bu durumda **commutative (değişimli) değildir**:

"Hello" + " World" → "Hello World"

" World" + "Hello" → " WorldHello" — farklı sonuç!

Ayrıca kullanıcı tanımlı operator overloading'de programcı +'ı commutative olmayan şekilde tanımlayabilir.

P8. Describe a situation in which the add operator would not be associative.

Cevap

Kayan nokta aritmetiğinde çok büyük/küçük sayılarla çalışırken toplama associative olmayabilir:

Overflow örneği: A ve C çok büyük pozitif, B ve D çok büyük negatif:

- $(A + B) + (C + D) \rightarrow$ taşma olmayabilir
- $(A + C) + (B + D) \rightarrow$ taşma olabilir!

Java'da string birleştirme: $(1 + 2) + "3" \rightarrow "33"$

$1 + (2 + "3") \rightarrow "123"$ — farklı sonuç!

P9. Show the order of evaluation with the given precedence/associativity rules.

Verilen kurallar (soldan sağa):

Öncelik	Operatörler
En Yüksek	*, /, not
2	+, -, &, mod
3	- (unary)
4	=, / =, <, < =, > =, >
En Düşük	and, or, xor

(a) $a * b - 1 + c$

1. $a * b$ (en yüksek: *) 2. $(a*b) - 1$ (soldan sağa) 3. $((a*b)-1) + c$

Gösterim: $((a * b)^1 - 1)^2 + c)^3$

(b) $a * (b - 1) / c \text{ mod } d$

1. $(b-1)$ (parantez) 2. $a*(b-1)$ 3. $(a*(b-1))/c$ 4. $((a*(b-1))/c) \text{ mod } d$

(c) $(a-b)/c \ \& \ (d*e/a-3)$

1. $(a-b)$ (parantez) 2. $d*e$ 3. $(d*e)/a$ 4. $((d*e)/a)-3$ 5. $(a-b)/c$ 6. $((a-b)/c) \ \& \ (((d*e)/a)-3)$

(d) $\neg a$ or $c = d$ and e Öncelik: unary $\neg$ > relational $=$ > and > or 1. $(\neg a)$ 2. $(c = d)$ 3. $((c = d) \text{ and } e)$ 4. $(\neg a) \text{ or } ((c = d) \text{ and } e)$ **(e) $a > b$ xor c or $d \leq 17$** 1. $(a > b)$ 2. $(d \leq 17)$ 3. $((a > b) \text{ xor } c)$ 4. $((a > b) \text{ xor } c) \text{ or } (d \leq 17)$ **(f) $\neg a + b$** 1. $(\neg a)$ (unary minus) 2. $(\neg a) + b$ Gösterim: $((\neg a)^1 + b)^2$ **P10. Show the order assuming no precedence, right-to-left associativity.****Cevap**

Öncelik yoksa ve tüm operatörler sağdan sola değerlendirilirse, en sağdaki operatör her zaman önce çalışır. Parantezli kısımlar her durumda önce tamamlanır.

(a) $a * b - 1 + c$: $\rightarrow (a * (b - (1 + c)^1)^2)^3$ **(b)** $a * (b - 1) / c \bmod d$: Parantez önce; kalan sağdan sola: $a * ((b - 1) / (c \bmod d))$ **(c)–(f)**: Benzer mantıkla her zaman sağdaki operatör önce.**P11. Write a BNF description of the precedence and associativity rules.****Cevap**

Sol özyinelemeli kurallar sol associativity'yi sağlar. Her katman bir öncelik düzeyine karşılık gelir:

```

1 <expr>      -> <expr> or  <and_expr>
2             | <expr> xor <and_expr>
3             | <and_expr>
4 <and_expr>   -> <and_expr> and <rel_expr>
5             | <rel_expr>
6 <rel_expr>   -> <add_expr> <relop> <add_expr>
7             | <add_expr>
8 <relop>      -> = | /= | < | <= | >= | >
9 <add_expr>   -> <add_expr> + <mul_expr>
10            | <add_expr> - <mul_expr>
11            | <add_expr> & <mul_expr>
12            | <add_expr> mod <mul_expr>
13            | <mul_expr>
14 <mul_expr>   -> <mul_expr> * <unary_expr>
15            | <mul_expr> / <unary_expr>
16            | <mul_expr> not <unary_expr>
17            | <unary_expr>

```



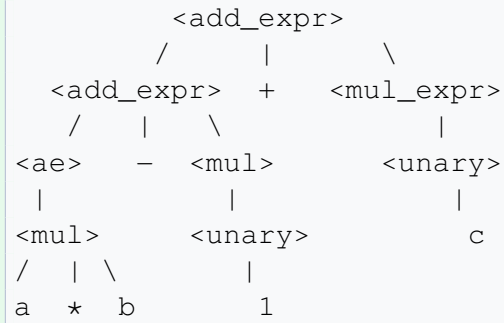
```

18 <unary_expr> -> - <primary> | <primary>
19 <primary>    -> a | b | c | d | e

```

P12. Draw parse trees for the expressions of Problem 9.

Cevap — (a) $a * b - 1 + c$



Sol özyinelemeli yapı, $((a * b) - 1) + c$ gruplama sırasını gösterir.

P13. Let fun be: `int fun(int* k) { *k += 4; return 3*(*k)-1; }`. What are sum1 and sum2?

Cevap

```
i = 10, j = 10    sum1 = (i/2) + fun(&i);    sum2 = fun(&j) + (j/2);
```

(a) Soldan sağa:

- **sum1:** Önce $i/2 = 5$, sonra `fun(&i)`: $i=14$, döner $3 \times 14 - 1 = 41 \Rightarrow \text{sum1} = 5 + 41 = 46$
- **sum2:** Önce `fun(&j)`: $j=14$, döner 41, sonra $j/2 = 7 \Rightarrow \text{sum2} = 41 + 7 = 48$

(b) Sağdan sola:

- **sum1:** Önce `fun(&i)`: $i=14$, döner 41, sonra $i/2 = 7 \Rightarrow \text{sum1} = 7 + 41 = 48$
- **sum2:** Önce $j/2 = 5$, sonra `fun(&j)`: $j=14$, döner 41 $\Rightarrow \text{sum2} = 41 + 5 = 46$

Yorum: Operand sırası sonucu doğrudan etkiler. C bu sırayı tanımlamadığı için derleyiciye göre değişebilir!

P14. What is your primary argument against the operator precedence rules of APL?

Cevap

Aleyhine ana argüman: APL'nin tek düzey öncelik + sağdan sola associativity tasarımı **matematiksel sezgiye aykırıdır**. $A * B + C \rightarrow A * (B + C)$ olarak değerlendirilir — oysa matematik $(A * B) + C$ der. Diğer dillerden gelen programcıların APL kodunu okurken sürekli zihinsel ek çaba harcamasına neden olur.

Lehine argüman: Tek ve tutarlı kural — ezberlenecek öncelik tablosu yoktur. Ancak bu avantaj, matematiksel sezgiyle çelişmenin maliyetini karşılamaz.

P15. Explain why it is difficult to eliminate functional side effects in C.

Cevap

C yalnızca fonksiyon yapısını destekler ve fonksiyonlar **tek bir değer döndürür**. Birden fazla değer iletmek için:

- **Pointer parametreler** zorunludur → bu side effect'tir.
- **Global değişkenlere erişim** verimlilik için zorunludur (örn. derleyicilerdeki sembol tablosu).
- Struct ile döndürmek mümkün olsa da okunurluk ve performans olumsuz etkilenebilir.

Sonuç: C'de side effect'leri tamamen yasaklamak dilin temel kullanım kalıplarını işlevsiz kılar.

P16. Make up a list of operator symbols to eliminate all operator overloading.

Cevap (Java için)

Operasyon	Mevcut	Önerilen
Tam sayı toplama	+	+i
Float toplama	+	+f
String birleştirme	+	^^
Tam sayı çarpma	*	*i
Float çarpma	*	*f

Bu yaklaşım okunurluğu ciddi şekilde düşürür; pratikte benimsenmez.

P17. Determine whether narrowing explicit type conversions provide error messages.

Cevap

Java: Narrowing cast için derleme zamanı uyarısı **vermez**; programcı açıkça cast yazmak zorundadır. Çalışma zamanında da hata mesajı yoktur — değer sessizce kesilir.

Python: `int(3.7)` → 3 (kesir atılır, uyarı yok). Ancak `int("abc")` → `ValueError` exception!

P18. Should an optimizing compiler for C or C++ be allowed to change the order of subexpressions in a Boolean expression?

Cevap

Hayır, olmamalıdır.

C/C++'ta `&&` ve `||` **short-circuit** semantiği garanti eder. Derleyici sıralamayı değiştirirse bu garantiyi bozar.

Kritik örnek: `p != NULL && p->val > 0` — sıralama değişirse null pointer hatası! Bir operandın side effect'i varsa (`++` veya fonksiyon çağırısı), sıralama sonucu

değiştirebilir. **Kural:** Optimizasyon, **semantiği değiştirmedığı** sürece yapılabilir.

P19. Consider: `int fun(int *i) {*i += 5; return 4;} void main() {int x = 3; x = x + fun(&x);}`

Cevap

(a) Soldan sağa:

- `x` okunur: 3
- `fun(&x)` çağrılır: `x = 8`, döner 4
- `x = 3 + 4 = 7`

(b) Sağdan sola:

- `fun(&x)` çağrılır: `x = 8`, döner 4
- `x` okunur: 8
- `x = 8 + 4 = 12`

P20. Why does Java specify left-to-right operand evaluation order?

Cevap

Platform bağımsızlığı ve deterministik davranış için. Java'nın temel hedefi: "Write Once, Run Anywhere." Aynı kod tüm JVM implementasyonlarında aynı sonucu üretmeli. Side effect içeren expression'ların tahmin edilebilir davranması için belirli bir sıra zorunludur. **Trade-off:** Bazı derleyici optimizasyonları imkânsız kılınır — ancak Java güvenilirliği öne çıkarmıştır.

P21. Explain how the coercion rules of a language affect its error detection.

Cevap

Kapsamlı coercion kuralları olan dillerde (C, C++) derleyici, uyumsuz tipleri otomatik dönüştürür. Bu durum:

- **Gerçek yazım hatalarını gizler:** `d = b * a` yerine `d = b * c` yazılması gerekirken `int a` otomatik coerce edilir → derleyici hata vermez!
- **Tip kontrol etkinliğini azaltır:** Tip sisteminin temel amacı olan hata tespiti zayıflar.

Ada ve ML'de coercion yoktur; tip uyumsuzluğu **derleme zamanı hatası** olarak yakalanır ve güvenilirlik artar.

CSE 341 · Gebze Technical University · Spring 2026